

Contenido

Guía del profesor — Laboratorio de IA y Visión por Computadora	1
1. De qué trata el laboratorio	1
2. Enfoque pedagógico (lo más importante)	2
3. Conceptos de fondo (para repasar)	2
3.1 Red neuronal convolucional (CNN)	2
3.2 Data augmentation	2
3.3 Transfer learning (Parte 2)	2
3.4 Clases del modelo: 4 (con fondo)	3
3.5 ADS / Beckhoff / pyads (Parte 2)	3
4. Cómo guiar la sesión	3
5. Errores frecuentes	3

Guía del profesor — Laboratorio de IA y Visión por Computadora

Documento interno (rama instructores). No se entrega al alumno. Explica de qué trata el laboratorio, cómo evaluarlo y los conceptos de fondo, por si no se dominan del todo.

1. De qué trata el laboratorio

El alumno entrena un clasificador de imágenes y lo conecta a un PLC **Beckhoff** para accionar un brazo robótico según el color detectado (control de calidad automatizado). Se divide en dos partes:

- **Parte 1 — IA (CNN).** Fundamentos: entrenar una red convolucional *desde cero*, ver que sobreajusta y generaliza mal, y mitigarlo con *data augmentation*. Termina aplicando la CNN a un dataset propio de tapitas.
- **Parte 2 — Visión (Transfer Learning + Beckhoff).** Reusa el dataset de tapitas, pero entrena con *transfer learning* (mejor modelo con pocas imágenes) y envía la predicción al PLC por el protocolo ADS.

Notebooks (carpeta parte1_ia/ y parte2_vision/):

#	Notebook	Qué hace
1	notebook1_cnn_desde_cero	CNN básica con botellas (vidrio/plástico)
2	notebook2_data_augmentation	Mostrar y mitigar el overfitting
3	notebook3_clasificador_tapitas	Dataset propio (videos → frames) + CNN
4	notebook4_transfer_learning	MobileNetV2 sobre las tapitas
5	notebook5_probar_modelo	Probar el modelo (imagen / cámara)
6	notebook6_envio_beckhoff	Cámara → predicción → 3 BOOLS por ADS

La nota es **/20**: Parte 1 (/10) + Parte 2 (/10). La rúbrica está en la Ficha de Evaluación (fe_lab.tex / fe_lab.pdf).

2. Enfoque pedagógico (lo más importante)

Los notebooks **ya están implementados y se ejecutan casi solos**. Por eso la nota **NO** va por “ejecutar y adjuntar la salida”, sino por **comprensión**:

- Responder las **preguntas conceptuales** intercaladas en cada notebook.
- **Interpretar** los resultados (¿por qué estas curvas?, ¿qué dice la matriz de confusión?, ¿por qué falla aquí?).
- **Justificar** decisiones (¿por qué este augmentation?, ¿por qué congelar el backbone?, ¿por qué un umbral de confianza?).

Como profesor, al revisar/preguntar, busque que el alumno **explique con sus palabras**, no que repita el código. Las preguntas de los notebooks son de nivel sencillo-medio y sirven de guion para la conversación oral.

3. Conceptos de fondo (para repasar)

3.1 Red neuronal convolucional (CNN)

Una CNN clasifica imágenes aprendiendo **filtros** que detectan patrones (bordes, texturas, formas). Capas clave: - **Convolución**: aplica filtros que recorren la imagen → “mapas de características”. Los filtros se aprenden solos durante el entrenamiento. - **Pooling**: reduce la resolución conservando lo importante (robustez + menos cómputo). - **Densas + softmax**: combinan las características y dan una probabilidad por clase. - **ReLU**: activación no lineal; sin ella la red sería una simple operación lineal.

Entrenamiento: se compara la predicción con la etiqueta real (*función de pérdida*), y un optimizador (*Adam*) ajusta los filtros por *backpropagation*, repitiendo sobre épocas y *batches*.

Overfitting: el modelo memoriza el set de entrenamiento (accuracy alta) pero falla en datos nuevos (accuracy de validación baja). Se diagnostica cuando las dos curvas se separan.

3.2 Data augmentation

Generar variaciones de las imágenes (rotación, brillo, zoom, desplazamiento) para que el modelo generalice mejor. **Solo se aplica a entrenamiento**, nunca a validación (la validación debe reflejar datos reales).

3.3 Transfer learning (Parte 2)

En vez de entrenar desde cero, se reutiliza **MobileNetV2**, ya entrenada sobre millones de imágenes (ImageNet). Idea clave: las **primeras capas** detectan cosas genéricas (bordes/texturas) útiles para casi cualquier imagen; solo las **últimas** son específicas de la tarea. Dos fases: - **Feature extraction**: se *congela* el backbone y se entrena solo un cabezal nuevo para las clases de tapitas. Rápido y efectivo. - **Fine-tuning**: se *descongelan* las últimas capas y se reentrena con un *learning rate* muy pequeño para afinar sin borrar lo aprendido.

Resultado: buen clasificador con pocos cientos de imágenes (la CNN desde cero necesitaría miles). El contraste NB3 (desde cero) vs NB4 (transferencia) sobre el **mismo** dataset es el punto pedagógico central.

3.4 Clases del modelo: 4 (con fondo)

El dataset tiene amarillo, azul, rojo y fondo (imágenes sin tapita). La clase fondo es **defensiva**: evita que el modelo fuerce un color cuando no hay objeto; **no acciona ninguna salida** del PLC.

3.5 ADS / Beckhoff / pyads (Parte 2)

- **ADS** (*Automation Device Specification*): protocolo de Beckhoff para leer y escribir variables del PLC **por su nombre** (no por dirección de memoria), sobre TCP/IP. TwinCAT es el entorno del PLC.
- **pyads**: librería de Python para hablar ADS. Se conecta con `Connection(AMS_NET_ID, 851)` y escribe con `write_by_name('VARIABLES.bRojo', True, pyads.PLCTYPE_BOOL)`.
- **Contrato (3 BOOLs)**: el clasificador activa bRojo/bAzul/bAmarillo según el color detectado (si supera el umbral de confianza); el resto en False. El PLC detecta el flanco y dispara el brazo.
- **HMI en TwinCAT**: el alumno arma una visualización con una **lámpara por color**; sirve para ver, sin instrumentos, que Python escribe la variable correcta. (Esto se hace en TwinCAT, no en el notebook.)
- **Ruta ADS**: error frecuente Missing ADS routes (7) → falta crear la *route* entre la PC y el IPC (TwinCAT → Router → Edit Routes → Add Route).

4. Cómo guiar la sesión

1. **Entorno (clave)**: que cada pareja cree el venv en VS Code con **Python 3.10/3.11**; con otra versión TensorFlow falla. VS Code instala `requirements.txt` al crear el entorno.
2. **Parte 1**: dejar que la CNN “falle” en datos reales (NB1/NB3) — ese fracaso motiva el transfer learning. No adelantar la solución.
3. **Parte 2**: el entrenamiento es casi automático (pesa poco en la nota). El foco está en la **comunicación real con Beckhoff** y la **inferencia en vivo**. Probar primero el envío de booleanos (NB6 etapa 3) antes de la cámara.
4. **Preguntar en voz alta**: usar las preguntas de los notebooks para verificar comprensión, no solo revisar capturas.

5. Errores frecuentes

Síntoma	Causa / solución
No module named tensorflow	Kernel/entorno equivocado; seleccionar el .venv correcto.
TensorFlow no instala	Python 3.12+ o del sistema; usar 3.10/3.11.

Síntoma	Causa / solución
Missing ADS routes (7)	Falta la route ADS PC ↔ IPC (TwinCAT → Router → Edit Routes).
AdsError: symbol not found	Nombre de variable no coincide con la GVL (mayúsculas, prefijo VARIABLES.).
Cámara no abre en WSL	WSL no accede a la cámara USB; ejecutar desde Windows.
Predice siempre lo mismo / mal	Preprocesamiento distinto al del entrenamiento, o dataset poco diverso.
Nada se envía al PLC	[SIM] = no conectado (revisar AMS Net ID y route), o confianza < umbral.